



ER 모델 연습 1

Scenario 1 — 대학교 수강신청 시스템

1. Scenario 이해

대학교 수강신청 시스템을 설계하세요.
학생은 수업을 신청하고, 교수는 수업을 가르칩니다.

2. Entity 도출

이 시스템에 필요한 개체(Entity)를 나열하세요.

Answer:

1. Student (학생)
 2. Professor (교수)
 3. Course (과목)
 4. Enrollment (수강) student_id (FK), course_id (FK)
 - 5.
-

3. M:N 관계 찾기

다대다(M:N) 관계가 있는가? 있다면 무엇인가?

Answer: 네 있지만 SQL에서 이런 관계 만들 수 없습니다. 그래서, 새로운 테이블 필요할 것 같아요.

Relationship:

Why M:N:

- 학생 ⇔ 교수 (m:1)
 - 교수 ⇔ 과목 (1:m)
 - 학생 ⇔ 과목 (m:n)
-

4. 새로운 개체 생성

M:N 관계를 해결하기 위해 새로운 개체를 설계하세요.

Answer:

New Entity Name:

Purpose:

5. Attributes 정의

(1) Student

Entity: Student

Attribute | Description

student_id	학번
name	이름
major	전공

👉 Primary Key:

PK:

(2) Professor

Entity: Professor

Attribute | Description

employee_id	사번
student_id(FK)	학번
name	이름
department	학과

👉 Primary Key:

PK:

(3) Course / Section

Entity: Course / Section

Attribute | Description

course_id	과목번호
section_id	분반번호
name	과목명

👉 Primary Key:

PK: 복합기 = **course_id + section_id**

(4) Enrollment (수강)

Entity: Enrollment

Attribute | Description

grade | 성적
enrolled_at | 신청일
|

👉 Primary Key (중요!):

PK: 복합키: **student_id (FK) + course_id (FK)**

6. 관계 정의

Entity A | Entity B | Relationship | Type (1:1, 1:N, M:N)

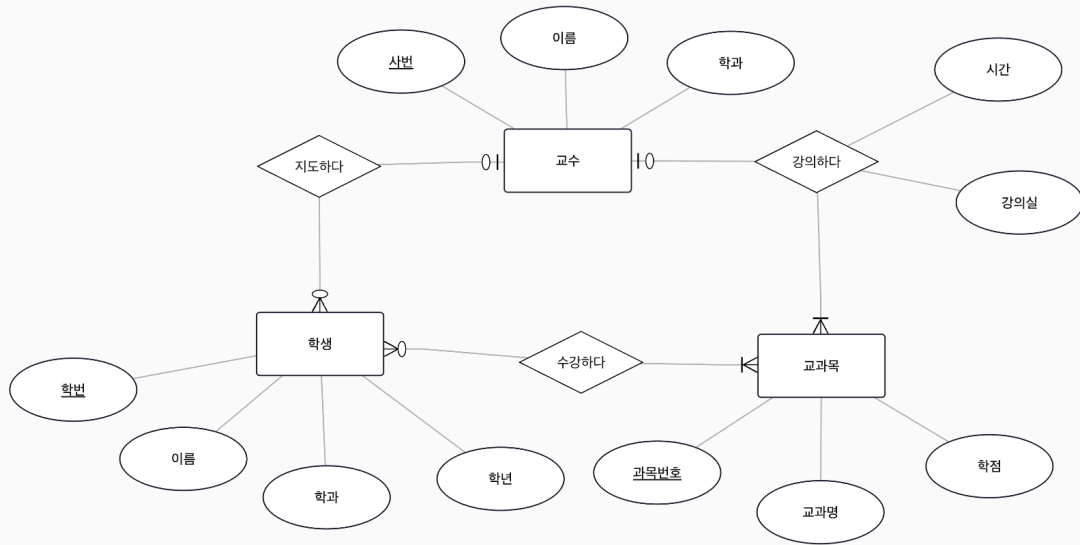
학생 ⇔ 교수 (m:1)
교수 ⇔ 과목 (1:m)
학생 ⇔ 과목 (m:n)
학생 ⇔ 수강 (1:m)
수강 ⇔ 과목 (m:1)

7. 카디널리티

각 관계의 (min, max)를 정의하세요.

- 학생 -> 교수
 - 교수 -> 학생
 - 학생 -> 수강
 - 수강 -> 학생
 - 과목 -> 수강
 - 수강 -> 과목
 - 교수 -> 과목
 - 과목 -> 교수
-

8. ER 다이어그램 만들



9. ER → SQL 변환

ER 모델을 기반으로 SQL 테이블을 설계하세요.

 Copy-paste SQL (실습):

```
DROP TABLE Professor;

CREATE TABLE Professor (
    professor_id int PRIMARY KEY,
    professor_name varchar(100),
    department varchar(100),
    salary numeric,
    salary_level numeric,
    hire_date date
);

CREATE TABLE Student (
    student_id int PRIMARY KEY,
    student_name varchar(100),
    major varchar(100)
);

'''
COURSE      | SECTION
```

```
course_id | 과목번호
section_id | 분반번호
name      | 과목이름
'''
```

```
CREATE TABLE Course (
    course_id int,
    section_id int,
    professor_id int,
    course_name varchar(100),
    PRIMARY KEY (course_id, section_id), -- 복합키
    FOREIGN KEY (professor_id) REFERENCES Professor (professor_id)
);
```

```
'''
수강
grade          | 성적
enrolled_at    | 신청일
student_id + course_id | 복합키
'''
```

```
CREATE TABLE Enrollment (
    student_id int,
    course_id int,
    grade varchar(2),
    points numeric, -- 99.65
    enrolled_at DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id)
    -- FOREIGN KEY (course_id) REFERENCES Course(course_id) -- Course의 복합키 땀에 오류
);
```

```

1  TABLE Professor {
2      professor_id int [pk]
3      professor_name varchar
4      department varchar
5      salary numeric
6      salary_level numeric
7      hire_date date
8  }
9
10 TABLE Student {
11     student_id int [pk]
12     student_name varchar
13     major varchar
14 }
15
16 TABLE Course {
17     course_id int
18     section_id int
19     professor_id int
20     course_name varchar
21     indexes {
22         (course_id, section_id) [pk]
23     }
24 }
25
26 TABLE Enrollment {
27     student_id int
28     course_id int
29     grade varchar
30     points numeric
31     enrolled_at date
32     indexes {
33         (student_id, course_id)
34     }
35 }
36
37 Ref: Enrollment.student_id > Student.student_id
38 Ref: Course.professor_id > Professor.professor_id
39
40

```

